# SOFTWARE REQUIREMENTS SPECIFICATION

for

# MATES

**A Taste Based Social Discovery Platform**

**Version 1.0**

**Prepared by :**
**1. Tulika Jain (1024160110)**
**2. Mukul (1024160124)**
**3. Pranav Tiwari (1024160127)**

**Submitted to : Anushka Ma'am**

**Course : UCS503P, Software Engineering Project**

**Computer Science and Engineering Department**
**Thapar Institute of Engineering and Technology, Patiala**

**October 15, 2026**

# Contents


**1 Introduction** **4**
- 1.1 Purpose . . . 4
- 1.2 Intended Audience and Reading Suggestions . . . 4
- 1.3 Project Scope . . . 4
- 1.4 Objectives . . . 5
- 1.5 Definitions, Acronyms and Abbreviations . . . 5
- 1.6 References . . . 6

**2 Overall Description** **7**
- 2.1 Product Perspective . . . 7
- 2.2 Stakeholders . . . 7
- 2.3 User Classes and Characteristics . . . 7
- 2.4 Product Functions . . . 8
- 2.5 Operating Environment . . . 8
- 2.6 Design and Implementation Constraints . . . 9
- 2.7 Assumptions and Dependencies . . . 9

**3 System Features** **10**
- 3.1 Description and Priority . . . 10
- 3.2 Functional Requirements . . . 10
  - 3.2.1 Authentication and Onboarding . . . 10
  - 3.2.2 Taste Import and Taste Signature . . . 11
  - 3.2.3 Compatibility Engine . . . 11
  - 3.2.4 Discovery . . . 11
  - 3.2.5 Connections and Chat . . . 12
  - 3.2.6 Match Breakdown . . . 12
  - 3.2.7 Content Drops . . . 12
  - 3.2.8 Communities . . . 13
  - 3.2.9 Safety, Privacy, and Preferences . . . 13

**4 External Interface Requirements** **14**
- 4.1 User Interface . . . 14
- 4.2 Software Interfaces . . . 14
- 4.3 Communications Interfaces . . . 14

**5 Other Nonfunctional Requirements** **15**
- 5.1 Performance Requirements . . . 15
- 5.2 Security Requirements . . . 15

5.3 Privacy Requirements . . . 15
5.4 Reliability and Availability . . . 16
5.5 Usability Requirements . . . 16
5.6 Maintainability . . . 16
5.7 Portability . . . 16
5.8 Scalability . . . 17

**6 System Diagrams** **18**
6.1 Use Case Diagram . . . 18
6.2 Context Diagram (DFD Level 0) . . . 18
6.3 Activity Diagram . . . 20
6.4 Class Diagram . . . 21
6.5 Sequence Diagram . . . 22

**7 Other Requirements** **23**
7.1 Business Rules . . . 23
7.2 Evaluation Metrics . . . 23
7.3 Legal and Attribution . . . 23

**8 Out of Scope and Future Work** **24**

# 1 Introduction

## 1.1 Purpose

Most people's taste is spread across separate apps: one for music, one for films, one for books. None of them gives a unified picture of who a person is, and the follower-based models common on social platforms optimise for reach rather than compatibility. MATES is a cross-domain taste-matching application. It matches people on their combined taste in music, movies, and books, and it gives each match a concrete, explainable reason to start a conversation.

This Software Requirements Specification (SRS) defines what MATES must do and the qualities it must have. It is the reference for design, implementation, testing, and evaluation.

## 1.2 Intended Audience and Reading Suggestions

This SRS is intended for the project team (developers and testers), the project guide, and the course evaluators. Readers who want the big picture should read Chapters 1 and 2. Developers should read Chapters 3 to 5 in full. Testers should focus on Chapters 3 and 4, since every functional and non-functional requirement carries an identifier that can be traced to a test.

## 1.3 Project Scope

MATES creates a space where people can find others who share their taste across three domains: music, movies, and books. The system includes:

- A Flutter mobile client for iOS and Android. A web build is kept for verification on the development machine.
- A Node.js and TypeScript REST backend built on Fastify, with PostgreSQL for storage and PGlite for local development.
- A dependency-free compatibility engine that scores pairs of users and explains each score.
- Integrations with external taste data providers (MusicBrainz, iTunes, TMDB, Wikidata, Open Library) and with the Claude API for narratives and icebreakers.

Features that are out of scope for this release are listed in Chapter 6.

## 1.4 Objectives

1. **Core loop.** A user can sign up, import taste, view ranked matches, see why each match was made, connect, and chat, all against live external data.
2. **Explainability.** Every compatibility score shows the specific shared items and divergence highlights that produced it, rather than an opaque ranking.
3. **Ongoing engagement.** Content Drops, Communities, and chat extend the app beyond a single match event.

## 1.5 Definitions, Acronyms and Abbreviations

| | |
|---|---|
| **SRS** | Software Requirements Specification. |
| **OTP** | One-Time Password, sent to an email address or phone number to sign in. |
| **JWT** | JSON Web Token, used as the session credential after sign-in. |
| **Taste Signature** | A user's taste across music, movies, and books, built from the items they add. |
| **Taste Vector** | The numeric representation of a Taste Signature, used for scoring. |
| **Compatibility Score** | A value between 0 and 1 (shown as a percentage) that measures how well two users' tastes match. |
| **Cosine similarity** | A measure of overall taste shape, based on the angle between two taste vectors. |
| **Weighted Jaccard** | A measure of literal shared favourites, where each shared item is weighted by its inverse popularity. |
| **Style alignment** | A measure of overlap in era and mood. |
| **Taste DNA** | A five-axis radar profile derived from a user's data. It is never self-reported. |
| **Drop** | A short post built from one of the user's own taste items. |
| **Community** | A tag-based group that users can join or leave. |
| **Icebreaker** | A suggested opening message based on shared or divergent taste. |
| **Divergence highlight** | An item that one user loves and the other user has not encountered. |
| **PGlite** | An embedded PostgreSQL build compiled to WebAssembly, used for local development. |

## 1.6 References


- MATES project proposal, UCS503P Mid-Semester Evaluation, 2026.
- Anthropic. Claude API Documentation. https://docs.claude.com
- Google. Flutter Documentation. https://docs.flutter.dev
- Fastify Team. Fastify Documentation. https://fastify.dev
- PostgreSQL Global Development Group. PostgreSQL Documentation. https://www.postgresql.org/docs/
- ElectricSQL. PGlite: Embedded Postgres in WASM. https://pglite.dev
- MusicBrainz API Documentation. https://musicbrainz.org/doc/MusicBrainz_API
- The Movie Database (TMDB). TMDB API Documentation. https://developer.themoviedb.org/docs
- Internet Archive. Open Library Developer Center. https://openlibrary.org/developers
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.

# 2 Overall Description

## 2.1 Product Perspective

MATES is a new, self-contained product. It is not a component of an existing system. It replaces the fragmentation of keeping music, film, and book taste in separate apps with a single taste profile. It also replaces follower-based discovery with compatibility-based discovery, so that every suggested person comes with a reason.

Figure 6.2 shows the system context. The MATES server exchanges data with the user (through the mobile app), with the external taste data providers, and with the Claude API.

## 2.2 Stakeholders

- **End users seeking friends.** Want to find people who share their taste, with low-friction conversation starters.
- **End users seeking dates.** Want compatibility that is explainable and trustworthy, with strong safety controls.
- **End users with both intents.** Want discovery filtered by the intent they have chosen.
- **Development team.** Needs maintainable, tested code and clear contracts between the app and the server.
- **Project guide (Anushka Ma'am).** Checks conformance to the approved proposal and the quality of the engineering.
- **Course evaluators.** Assess a working end-to-end system with evidence of testing and evaluation.
- **External providers.** MusicBrainz, iTunes, TMDB, Wikidata, Open Library, and Anthropic provide data and AI services. They expect fair, rate-limited use of their APIs and correct attribution.

## 2.3 User Classes and Characteristics

MATES has three user classes:

- **Registered user.** Has a verified account, completes onboarding, imports taste, and uses discovery, connections, drops, communities, and chat. This is the only class with access to product features.
- **Synthetic profile.** Created by hand through the "Add a profile" tool to seed the matching pool. It takes part in matching like any registered user and is flagged as synthetic. It exists for cold-start mitigation and demonstrations.
- **Administrator.** Not supported in this release. Moderation is performed through the safety actions available to users and through direct database access by the team.

## 2.4 Product Functions

MATES provides the following functions:

1. Sign-in with a one-time code and session tokens.
2. Onboarding with name, age, city, bio, and intent.
3. Taste import and Taste Signature construction across music, movies, and books.
4. Compatibility scoring with explainable shared items and divergence highlights.
5. Discovery of compatibility-ranked profiles in a swipeable card stack.
6. Opt-in connection requests, with chat unlocked on acceptance.
7. Match breakdown with a score ring, per-domain scores, a Taste DNA radar, and a narrative.
8. Content Drops, reactions, and Communities.
9. Threaded chat with read receipts and icebreakers.
10. Safety controls (block, report, unmatch) and per-domain privacy settings.

Figure 6.1 shows the use cases.

## 2.5 Operating Environment

- **Client:** Flutter 3.47 and Dart 3.13 on iOS and Android. The layout must work at a phone width of about 380 px.
- **Server:** Node.js 20 or later, TypeScript, and Fastify 5 on a Linux host or container.
- **Database:** PostgreSQL in deployment. PGlite, which stores data under `server/.pgdata/`, in local development. The same SQL must run on both.
- **Network:** HTTPS between the client and the server. Outbound HTTPS from the server to the external providers and the Claude API.

## 2.6 Design and Implementation Constraints

- **C1 (Language split).** The client is Dart and the server is TypeScript. The app cannot import shared TypeScript types, so the contract is a generated OpenAPI document (`shared/openapi.json`), and the Dart models are written against it.
- **C2 (Zero-configuration operation).** Every taste domain must work without an API key. A key can improve quality but must never be required for a feature to work.
- **C3 (No third-party authentication).** Authentication is implemented in-house with OTP and JWT, so the system runs without external accounts.
- **C4 (Engine isolation).** The compatibility engine must have no database, HTTP, or framework dependencies, so it can be unit tested on its own.
- **C5 (Canonical pair ordering).** Each pair of users is stored once, with `user_a_id` less than `user_b_id`, enforced by a CHECK constraint.
- **C6 (Toolchain).** iOS and Android builds need a complete Xcode installation and an Android SDK on the development machine. Until these are installed, the web target is used for verification.
- **C7 (Development OTP delivery).** In development, OTP codes are written to the server log and prefilled in the app. No SMS or email provider is connected yet.

## 2.7 Assumptions and Dependencies

- MusicBrainz sometimes returns HTTP 503 under load. iTunes is the fallback for music.
- TMDB is used only when `TMDB_API_KEY` is configured. Otherwise, Wikidata SPARQL is used.
- The Claude API is used only when `ANTHROPIC_API_KEY` is configured. Otherwise, a code-based fallback generates narratives that name real shared titles.
- Users give honest taste input. Taste Signature values come from data, not from self-report, except for stated intent and domain priority.
- External provider terms of use, including rate limits and attribution, are respected.
- A pilot with real users, supplemented by synthetic profiles, is planned for the next stage. Results are not yet available.

# 3 System Features

## 3.1 Description and Priority

MATES has a set of core features and a set of supporting features. All of them are required for the product to deliver its objectives. The features are listed below in priority order, highest first:

1. **Compatibility matching.** The goal feature of the product. The engine scores a pair of users and explains the score.
2. **Taste import.** Builds the Taste Signature that all matching depends on.
3. **Discovery.** Presents ranked candidates in a card stack.
4. **Connections and chat.** Turn a match into a conversation through opt-in requests.
5. **Match breakdown.** Shows the reasoning behind a score through the Taste DNA radar and narrative.
6. **Content Drops and Communities.** Sustain engagement after the first match.
7. **Safety and privacy.** Block, report, unmatch, and per-domain visibility.

Each functional requirement carries a priority (Must, Should, or Could) and a status. The status is *Implemented*, *Partial* (works with a stated limitation), or *Planned*.

## 3.2 Functional Requirements

### 3.2.1 Authentication and Onboarding

- **FR-01** The system shall send a one-time code to an email address or phone number when the user requests sign-in. *(Must; Partial: development logs the code to the console.)*
- **FR-02** The system shall verify the submitted code and, on success, issue a JWT session. *(Must; Implemented.)*
- **FR-03** A one-time code shall expire after a fixed period and shall be invalid after its first use. *(Must; Implemented.)*
- **FR-04** The system shall limit verification attempts per identifier to resist brute-force attacks. *(Must; Implemented.)*

- **FR-05** The app shall restore a valid session at launch without asking the user to sign in again. *(Should; Implemented.)*
- **FR-06** The system shall collect name, age, city, bio, and intent (dating, friends, or both) during onboarding, and shall validate each field. *(Must; Implemented.)*

### 3.2.2 Taste Import and Taste Signature

- **FR-07** The system shall let the user search for items in each of the three domains and see live results from the external providers. *(Must; Implemented.)*
- **FR-08** The system shall let the user add an item to, or remove an item from, their Taste Signature. *(Must; Implemented.)*
- **FR-09** If the primary provider for a domain is unavailable, the system shall use the secondary provider for that domain. *(Must; Implemented.)*
- **FR-10** The system shall derive a Taste Vector from the user's items and compute a completeness value for each domain. *(Must; Implemented.)*
- **FR-11** The system shall let the user set a priority for each domain. The priority affects how that domain is weighted in scoring. *(Should; Implemented.)*

### 3.2.3 Compatibility Engine

- **FR-12** For a pair of users $(a, b)$, the engine shall compute score $= 0.45 \times \text{cosine} + 0.35 \times \text{weightedJaccard} + 0.20 \times \text{styleAlignment}$, clipped to the range $[0, 1]$. *(Must; Implemented.)*
- **FR-13** Each shared item shall be weighted by inverse popularity, so that a shared obscure item contributes more than a shared blockbuster. *(Must; Implemented.)*
- **FR-14** A domain shall contribute to the score only when both users have data in it. Its weight shall scale with data depth and with each user's stated priority. An unconnected domain shall not pull the score toward zero. *(Must; Implemented.)*
- **FR-15** Every score shall return the concrete shared items that produced it, plus divergence highlights. *(Must; Implemented.)*
- **FR-16** The engine shall cache computed pair scores and shall recompute a cached score when either user's taste changes. *(Should; Partial: per-request cache, with no nightly precompute job.)*

### 3.2.4 Discovery

- **FR-17** The system shall present candidate profiles as a ranked, swipeable card stack ordered by compatibility score. *(Must; Implemented.)*

- **FR-18** Ranking shall take the user's stated intent into account. *(Must; Implemented.)*
- **FR-19** The user shall be able to filter candidates by minimum match score and by age range. *(Must; Implemented.)*
- **FR-20** A pass action shall remove the card from the stack without creating any record that the other user can see. *(Must; Implemented.)*

### 3.2.5 Connections and Chat

- **FR-21** A like shall create a connection request to the liked user. Connections are opt-in and bidirectional. *(Must; Implemented.)*
- **FR-22** If the other user has already liked the first user, the connection shall be accepted immediately. *(Must; Implemented.)*
- **FR-23** The recipient shall be able to accept a pending request from the Matches tab. Requests that are not accepted remain pending. *(Must; Implemented.)*
- **FR-24** Chat between two users shall be enabled only after the connection status is *accepted*. *(Must; Implemented.)*
- **FR-25** Chat shall support threaded messages with read receipts. *(Must; Implemented.)*
- **FR-26** Chat shall offer taste-based icebreakers, generated from shared or divergent items. *(Should; Implemented.)*

### 3.2.6 Match Breakdown

- **FR-27** The breakdown screen shall show the overall score as a score ring, the per-domain scores, and a five-axis Taste DNA radar. *(Must; Implemented.)*
- **FR-28** The Taste DNA axes shall be derived from the user's data and shall never be self-reported. *(Must; Implemented.)*
- **FR-29** The breakdown shall include a narrative. The Claude API generates it when a key is configured. Otherwise, a code-based fallback generates it, and the fallback names real shared titles. *(Must; Implemented.)*
- **FR-30** The breakdown shall provide a share and differ toggle that switches between shared highlights and divergence highlights. *(Should; Implemented.)*

### 3.2.7 Content Drops

- **FR-31** The user shall be able to create a Drop from one of their own taste items, with an optional caption and an optional community tag. *(Must; Implemented.)*

- **FR-32** Users shall be able to react to a Drop with *like* or *save*. Reactions are stored per user. *(Must; Implemented.)*
- **FR-33** When a Drop matches an item in the viewer's own profile, the Drop shall show a "you have this too" flag. *(Should; Implemented.)*

### 3.2.8 Communities

- **FR-34** The system shall suggest Communities ordered by the overlap between the user's taste tags and each Community's tag. *(Must; Implemented.)*
- **FR-35** The user shall be able to join and leave a Community. *(Must; Implemented.)*
- **FR-36** Each Community shall show its member list and a feed of Drops tagged to it. *(Must; Implemented.)*

### 3.2.9 Safety, Privacy, and Preferences

- **FR-37** The user shall be able to block another user. A blocked user shall not appear in the blocker's discovery feed or be able to contact them. *(Must; Implemented.)*
- **FR-38** The user shall be able to report another user with a reason. Reports are stored for later review. *(Must; Implemented.)*
- **FR-39** The user shall be able to unmatch an existing connection. After unmatching, neither party shall be able to access the chat history. *(Must; Implemented.)*
- **FR-40** For each taste domain, the user shall set visibility to *full*, *aggregate*, or *hidden*. Other users shall see only the level that is permitted. *(Must; Implemented.)*
- **FR-41** The user shall be able to build a synthetic profile by hand. The profile shall join the matching pool immediately and shall be marked as synthetic. *(Could; Implemented.)*

# 4 External Interface Requirements

## 4.1 User Interface

- UI-01: The app shall present a five-tab bottom navigation: Discover, Drops, Rooms, Matches, and Taste.
- UI-02: The app shall use a dark theme by default. A light theme and a system-following option shall be available, and the choice shall persist across sessions.
- UI-03: The domain accent colours (music, movies, books) shall be used only to identify domains and primary actions, so that a colour always means the same thing.
- UI-04: Match scores shall be shown as a score ring and as numerals with tabular figures, so that the digits do not shift during animation.

## 4.2 Software Interfaces

- SI-01: The server shall expose a REST API described by a generated OpenAPI document. This document is the contract between the client and the server.
- SI-02: The server shall call MusicBrainz, iTunes, TMDB, Wikidata, and Open Library for taste lookup.
- SI-03: The server shall call the Claude API for narrative and icebreaker generation when the API key is configured.
- SI-04: The server shall access PostgreSQL (or PGlite in development) using parameterised SQL only.

## 4.3 Communications Interfaces

- CI-01: Client-server communication shall use HTTPS with JSON payloads in production.
- CI-02: Authenticated requests shall carry a JWT in the `Authorization` header.

# 5 Other Nonfunctional Requirements

## 5.1 Performance Requirements

- NFR-01: A discovery request shall return within 1 second when all required pair scores are cached. When scores must be computed, it shall return within 3 seconds at the 95th percentile, for a candidate pool of up to 500 users.
- NFR-02: A taste search shall return results from the external provider within 3 seconds, or fall back to the secondary provider.
- NFR-03: The score ring and Taste DNA radar shall render without visible jitter during animation.

## 5.2 Security Requirements

- NFR-04: OTP codes shall be stored hashed, never in plain text, and shall be single-use and time-limited.
- NFR-05: JWTs shall be signed and shall have an expiry. Protected endpoints shall reject requests with missing or invalid tokens.
- NFR-06: All database access shall use parameterised queries.
- NFR-07: The system shall apply rate limiting to OTP requests and verification attempts.
- NFR-08: API keys (Claude, TMDB) shall be read from environment variables and shall never be shipped inside the client application.
- NFR-09: All data in transit shall use HTTPS in production.

## 5.3 Privacy Requirements

- NFR-10: The server shall enforce per-domain privacy settings (FR-40) on every response that exposes taste data. Client-side enforcement alone is not sufficient.
- NFR-11: Users shall be able to request deletion of their account and data. This flow is planned and is listed in Chapter 6.

## 5.4 Reliability and Availability

- NFR-12: If the Claude API is unavailable or not configured, narratives and icebreakers shall still be produced by the fallback, without a user-visible error.
- NFR-13: If a music provider returns an error, the system shall use the fallback provider (FR-09).
- NFR-14: Database constraints shall reject invalid states, including an unordered pair, a self-connection, and a self-like.
- NFR-15: The deployed backend shall have a target availability of 99% per month. Measuring this requires a deployment target with monitoring, which is planned.

## 5.5 Usability Requirements

- NFR-16: A new user shall be able to complete sign-in, onboarding, and taste import without consulting external documentation.
- NFR-17: Each compatibility score shall be shown together with the shared items behind it, so the reason for a match is visible without extra navigation.
- NFR-18: The controls for blocking, reporting, and unmatching shall be reachable from any profile or chat.
- NFR-19: The app shall remain usable at a phone width of about 380 px, with no horizontal scrolling.

## 5.6 Maintainability

- NFR-20: The compatibility engine shall contain no database, HTTP, or framework dependencies (constraint C4).
- NFR-21: The server test suite (Vitest) shall cover authentication, compatibility, connections, drops, and communities.
- NFR-22: Static analysis (`flutter analyze`) shall report zero issues on the client.
- NFR-23: The OpenAPI document shall be regenerated from the server source with a single command.

## 5.7 Portability

- NFR-24: The same SQL shall run on PGlite (development) and on PostgreSQL (deployment).
- NFR-25: The client shall target iOS and Android. A web build shall remain available for verification.

## 5.8 Scalability

- NFR-26: The design shall allow a move from per-request score computation to a nightly precompute job, when the user volume justifies it, without changing the public API.

# 6 System Diagrams

## 6.1 Use Case Diagram

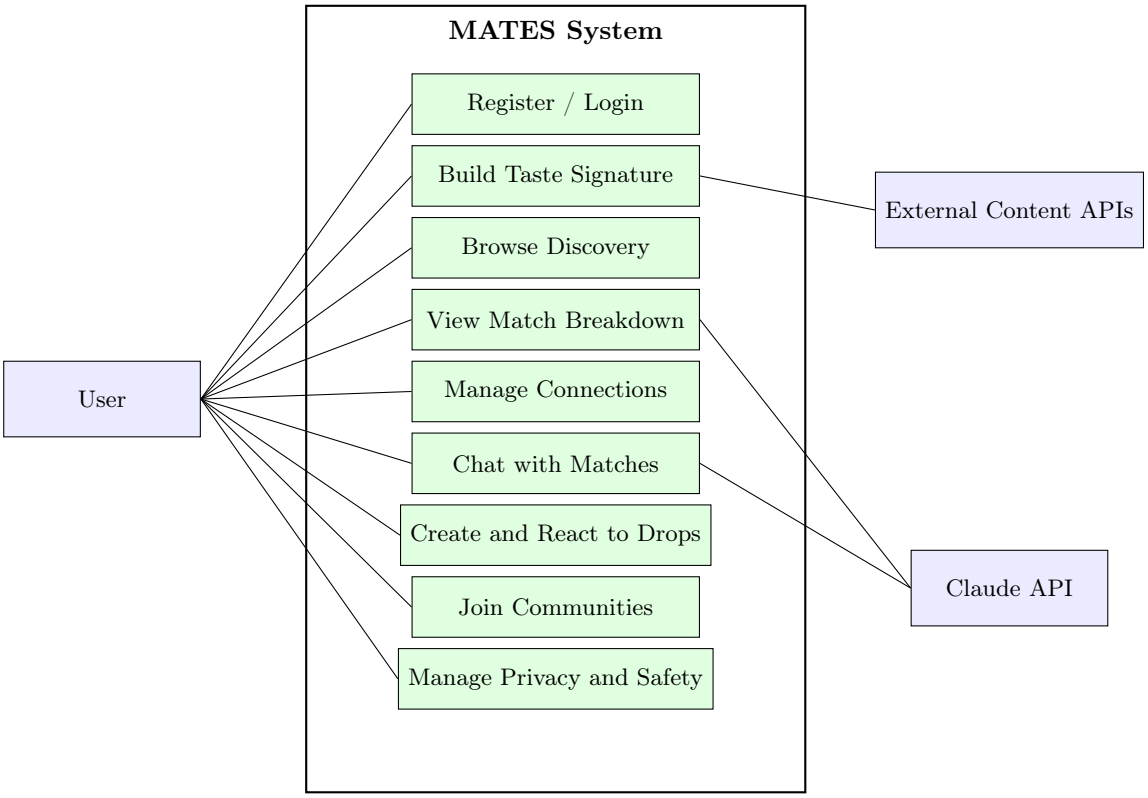



Figure 6.1: Use case diagram of the MATES system.

## 6.2 Context Diagram (DFD Level 0)

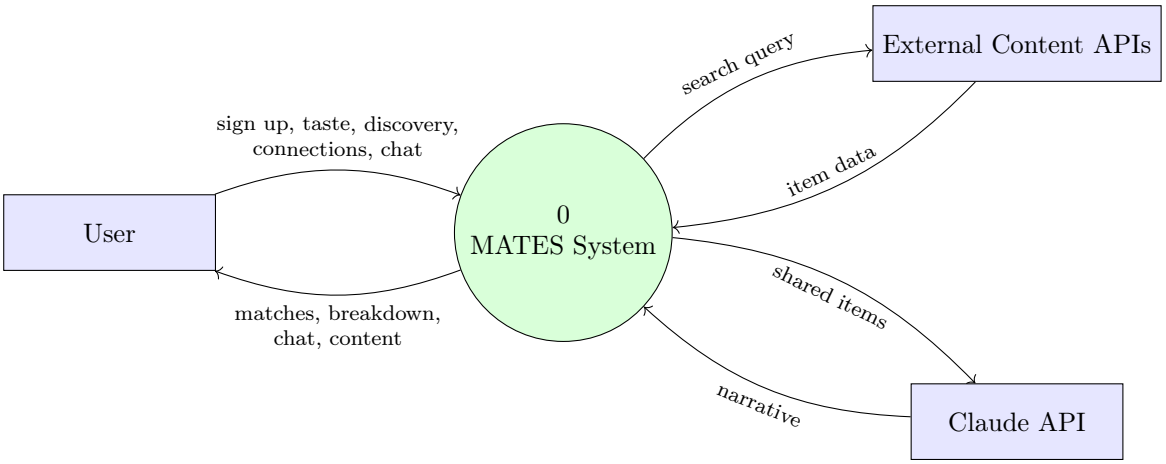



Figure 6.2: Data flow diagram, Level 0 (context diagram) of MATES.

## 6.3 Activity Diagram

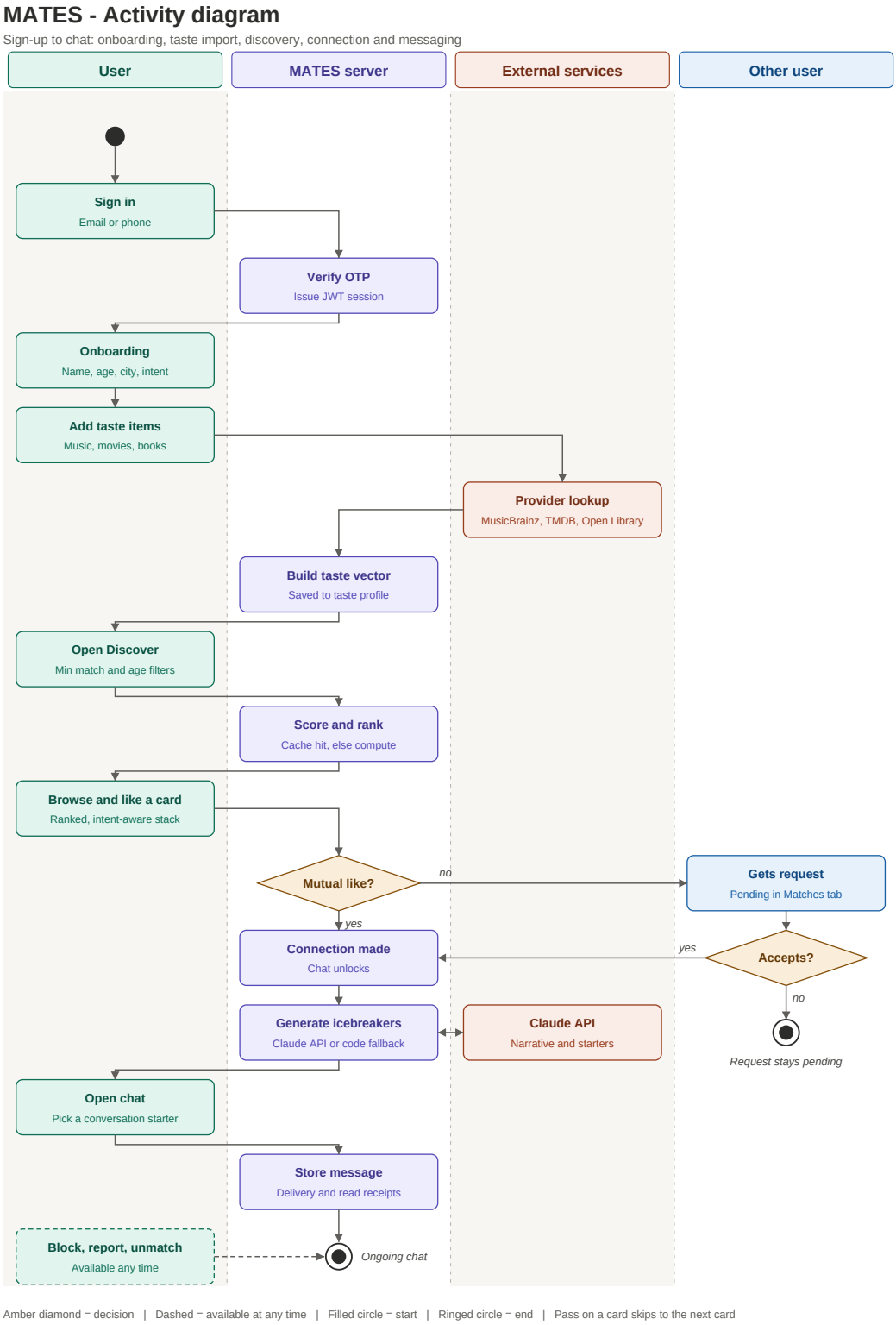



Figure 6.3: Activity diagram of the MATES sign-up-to-chat flow.

## 6.4 Class Diagram

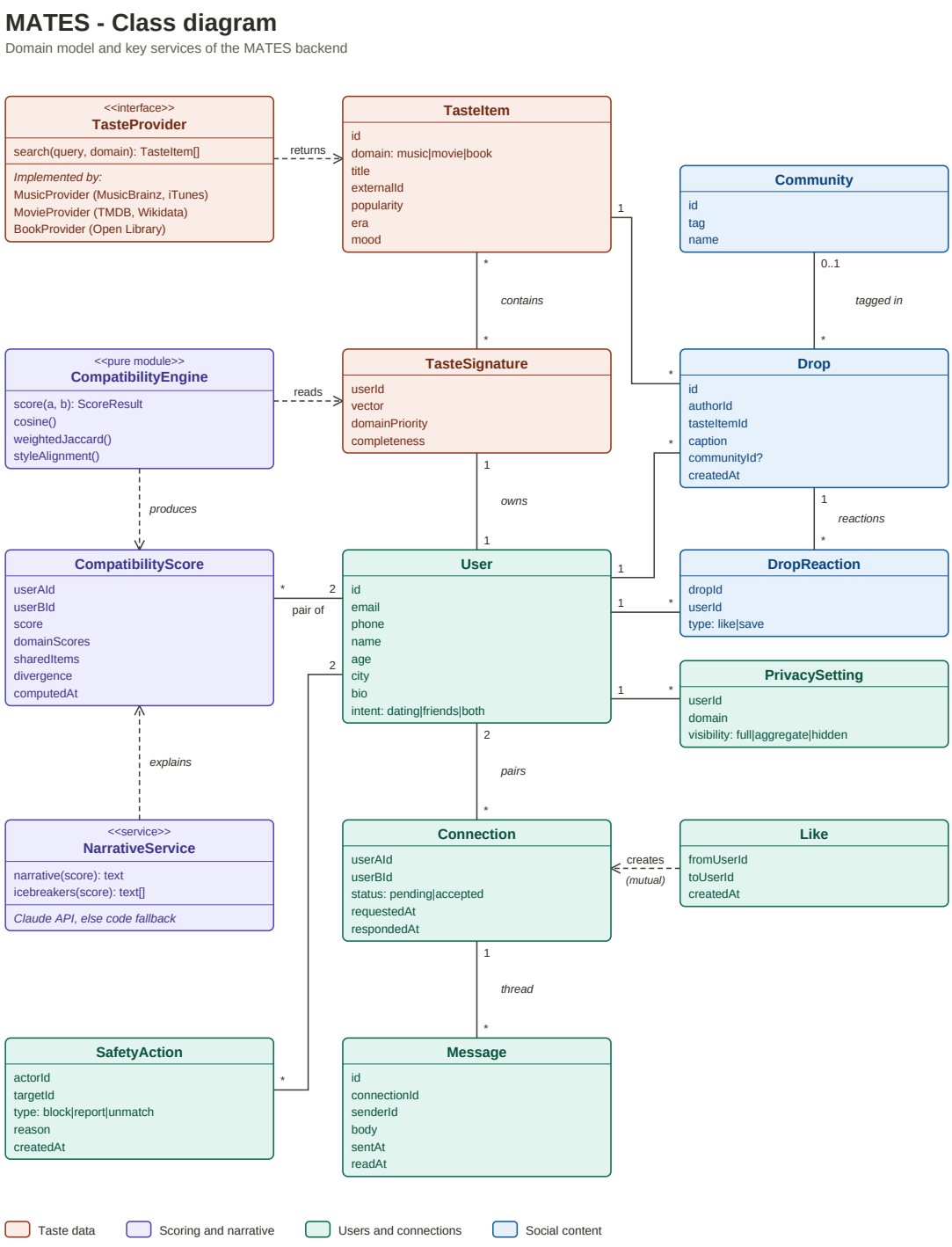



Figure 6.4: Class diagram of the MATES domain model and key services.

## 6.5 Sequence Diagram

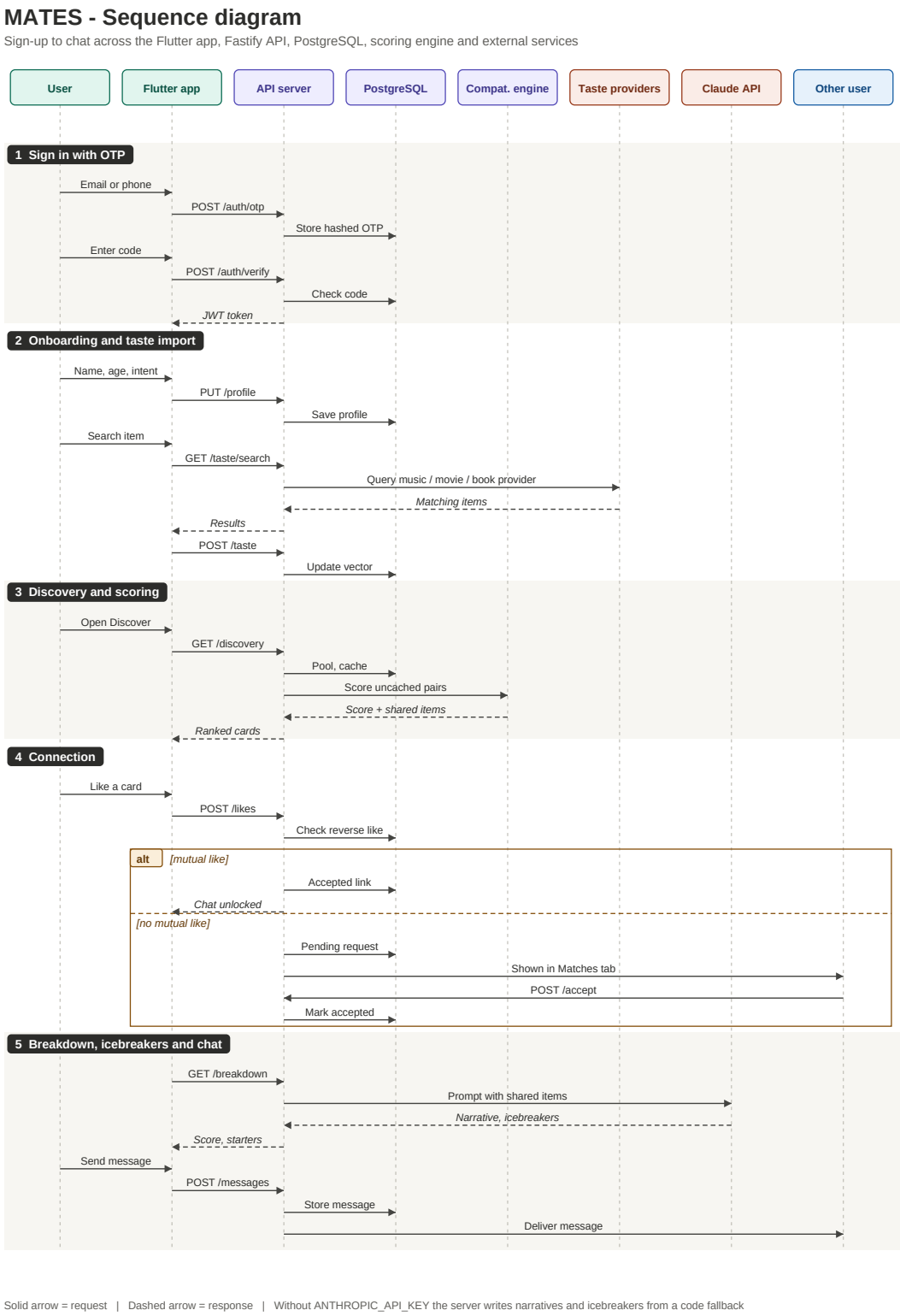



Figure 6.5: Sequence diagram of sign-up to chat across the app, API, database and external services.

# 7 Other Requirements

## 7.1 Business Rules

- BR-01: Pairs of users are scored symmetrically. The score for $(a, b)$ equals the score for $(b, a)$.
- BR-02: A connection moves to chat only after acceptance. A mutual like counts as acceptance.
- BR-03: Synthetic profiles are labelled as synthetic everywhere they appear.

## 7.2 Evaluation Metrics

The following metrics will be used to evaluate matching quality once the pilot runs. Their underlying events are already timestamped in the schema.

- Match-to-connection rate (instrumented).
- Connection acceptance rate and response time (instrumented).
- Taste Signature completion rate (instrumented).
- Match explanation density, that is, shared highlights stored per pair (instrumented).
- Explainability satisfaction on a 1 to 5 scale (not yet built; needs an in-app rating prompt).

## 7.3 Legal and Attribution

Taste data is shown with attribution to its provider. Artwork and catalogue data are used under each provider's terms of use.

# 8 Out of Scope and Future Work

The following features are not part of this release:

- Notification layer for connections, likes, and new matches.
- Photo upload pipeline. Profile portraits are currently generated, and the schema and moderation columns already exist.
- Spotify OAuth and audio-feature (sonic fingerprint) data.
- Taste Rooms, Blind Taste Mode, Collaborative Blends, and Weekly Recap. Schema support exists for Rooms and Blends.
- In-app search across users and content.
- Admin and moderation dashboard.
- SMS or email OTP delivery provider.
- CI/CD pipeline and production deployment target.
- Account deletion flow.